Institutional Blockchain Infrastructure Series | Paper III of V

Canonical Artifact Identity and Satoshi-Bound Provenance for Regulated Settlement Systems

Series

Institutional Blockchain Infrastructure Series — Paper III

Version

1.0 | July 2026

Classification

Public Distribution

Date

13 July 2026

Status

Final — Approved for External Release

Author Note

This paper is the third in the Institutional Blockchain Infrastructure Series. The first paper,

Foundational Tokenization Architecture for Institutional Assets

, established the structural primitives — UTXO semantics, scripting constraints, and institutional custody models — that underpin blockchain-native token issuance. The second paper,

Institutional Settlement Semantics and Atomic Transaction Finality

, extended that foundation into the settlement layer, formalizing the conditions under which atomic finality can be claimed and relied upon in regulated post-trade contexts. The present paper extends the series' treatment of token semantics into the identity and provenance layer: it defines how real-world artifacts become canonical, how their identity is cryptographically bound to satoshis — the atomic units of the UTXO ledger — and how this binding enables fully auditable, deterministically reconstructible settlement chains suitable for regulatory examination, cross-jurisdictional compliance, and long-horizon institutional record-keeping. Papers IV and V, forthcoming, will address custody infrastructure integration and cross-network settlement protocols respectively.

Abstract

Institutional tokenization of financial instruments, legal documents, and physical assets has advanced rapidly since the publication of foundational UTXO-native architecture frameworks. However, the dominant operational models continue to treat tokens primarily as representations of value — carriers of economic entitlement — without encoding the deterministic identity of the underlying artifact through every state transition the token undergoes. This omission creates critical failures in post-trade infrastructure: audit trail fragmentation, provenance ambiguity across counterparty chains, and reconstruction failure in insolvency or dispute resolution contexts where historical state must be recovered with cryptographic certainty.

This paper argues that institutional tokenization requires not merely value transfer but *deterministic identity transfer* — a guarantee that the cryptographic identity of the artifact being settled

accompanies every ledger event, every custody transition, and every regulatory disclosure throughout the asset's full lifecycle. We present five core contributions toward this requirement. **First**, we provide a formal definition of canonical artifact identity, grounded in the JSON Canonicalization Scheme (RFC 8785) and byte-stable serialization disciplines, establishing what it means for a digital artifact to have a single authoritative form. **Second**, we specify the satoshi-binding protocol — a mechanism for anchoring an artifact's canonical digest to individual satoshis within the UTXO set, exploiting their atomic, persistently traceable, and non-duplicable properties to create identity anchors that survive transfers, splits, and cross-custody movements. **Third**, we introduce the provenance envelope architecture — an append-only, cryptographically sealed metadata structure that accumulates the complete event history of an artifact from issuance through archival, enabling selective disclosure and Merkle-verified partial attestation. **Fourth**, we define deterministic replay and reconstruction semantics — a stateless protocol by which any authorized party can reconstruct the precise state of any artifact at any historical point in time from the ordered event log and canonical record alone. **Fifth**, we specify the lifecycle governance framework that governs state transitions across the artifact's full operational life within a regulated institutional environment.

Together, these contributions constitute a complete, institutionally viable identity and provenance layer for tokenized assets. The architecture directly addresses post-trade audit requirements under EMIR, SFTR, MiCA Articles 76–80, and SEC Rule 17a-4, and provides a neutral cryptographic identity substrate capable of operating across multiple legal jurisdictions without depending on any single regulatory authority for its validity. The paper's significance extends beyond compliance: it establishes the technical precondition for cross-network settlement interoperability and institutional custody integration treated in subsequent papers in this series.

1 Introduction

1.1 Motivation: The Identity Gap in Institutional Tokenization

The decade following the publication of the Bitcoin whitepaper in 2008 demonstrated that peer-to-peer electronic value transfer was possible without trusted intermediaries. The decade that followed demonstrated that the same ledger primitives — cryptographic commitment, UTXO lineage, and script-

enforced spending conditions — could be extended to represent not merely currency but complex financial instruments, legal rights, and real-world asset ownership. Institutional interest in blockchain-native tokenization has consequently intensified, with central banks, investment custodians, clearinghouses, and multilateral financial institutions publishing frameworks, pilots, and regulatory analyses at an accelerating pace.

Yet a structural problem has emerged in the operational implementation of institutional tokenization: existing approaches treat the token as a proxy for value, not as a carrier of deterministic artifact identity. A token issued to represent a trade confirmation, a bond instrument, or a real estate deed encodes the economic entitlement — ownership, par value, redemption rights — but does not encode, in a cryptographically verifiable and lifecycle-persistent manner, the precise question: *which version, which serialization, which canonical form of this artifact is authoritative at the moment of settlement?*

This omission has consequences that are not merely theoretical. In post-trade reconciliation, counterparties frequently maintain divergent representations of the same instrument, and the absence of a canonical identity reference makes automated reconciliation brittle. In regulatory examination, supervisory authorities require the ability to reconstruct the precise state of an instrument at a historical timestamp — a requirement that cannot be satisfied if the instrument's state was never deterministically anchored to an ordered, immutable event sequence. In insolvency proceedings, the administrator must establish with legal certainty which party held which instrument in which state at which time — a task that becomes tractable only if the instrument's identity history is cryptographically self-evident from the ledger alone.

The Financial Stability Board's 2023 report on tokenization in financial markets identified provenance and audit trail integrity as among the primary institutional barriers to widespread adoption. The BIS working paper on tokenization similarly flagged the absence of standardized identity anchoring as a gap in current technical specifications. This paper responds directly to that gap.

1.2 The Canonical Identity Problem

The core challenge can be stated precisely. When a financial instrument, legal document, or physical asset is tokenized, the tokenization event creates a correspondence between a real-world artifact and a ledger representation. For institutional purposes, this correspondence must satisfy a property we term

canonical uniqueness: at any point in time, there must be exactly one answer — cryptographically verifiable by any authorized party — to the question "what is the authoritative state of this artifact?"

This requirement is more demanding than it may initially appear. A trade confirmation document, for example, may exist simultaneously as: a PDF file in a trade management system, a structured JSON record in a messaging layer, a summarized entry in a central counterparty's database, a reference in a regulatory reporting repository, and a token on a distributed ledger. Each of these representations may differ in format, encoding, metadata inclusion, and field precision. Without a canonical identity standard, there is no neutral, computationally verifiable mechanism for establishing which of these representations is authoritative — and consequently no mechanism for asserting that a settlement event transferred the correct artifact.

The canonical identity problem is further complicated by the lifecycle of artifacts over time. A bond instrument may be amended multiple times between issuance and redemption. Each amendment creates a new version of the artifact, and the settlement system must not only identify the canonical form of each version but also maintain a complete, ordered, cryptographically verifiable history of the transition from one canonical version to the next. This is the provenance chain, and its integrity is the foundational requirement for regulated settlement.

1.3 Satoshi as Identity Anchor

A central proposition of this paper is that the satoshi — the smallest indivisible unit of value in the Bitcoin protocol, equal to 10^{-8} of a bitcoin — is the ideal identity anchor for real-world artifacts in institutional tokenization systems. This claim rests on four properties of satoshis that no other candidate identity anchor currently satisfies simultaneously.

Atomicity: Satoshis are indivisible at the protocol level. An output carrying a satoshi either exists in the UTXO set or it does not. There is no partial satoshi, no fractional UTXO. This atomicity makes satoshis uniquely suitable as discrete identity carriers — each satoshi can carry exactly one artifact identity, and that identity cannot be subdivided or blurred by protocol operations.

Persistent UTXO Lineage: Every satoshi in existence can be traced from its current UTXO back through every transaction in which it participated to its coinbase origin. This lineage is stored in the public ledger

and is accessible to any node. For identity anchoring purposes, this means the complete transfer history of the artifact is embedded in the public ledger state itself, independent of any off-chain system.

Non-Duplicability: The Bitcoin protocol's double-spend prevention guarantees that no satoshi can appear simultaneously in two unspent outputs. A satoshi carrying an artifact's identity is, at any moment, in exactly one UTXO. This non-duplicability is the physical impossibility of the artifact being simultaneously "owned" by two different parties at the ledger level.

Observability of State Changes: Any event that changes the custody or encumbrance of a satoshi — spend, lock, split-merge — is an observable ledger event with a block height timestamp. This means that state changes to the artifact's identity anchor are automatically timestamped and ordered by the global ledger consensus mechanism, without reliance on any single party's clock or record-keeping system.

Contrast this with account-based ledgers, where identity anchoring at this level of granularity is structurally impossible: an account balance does not have a traceable lineage to a specific origin; the same balance unit can be conceptually co-mingled with other units; and state changes are recorded as account balance adjustments rather than as observable movements of discrete, identifiable units.

1.4 Paper Structure

The remainder of this paper is organized as follows. Section 2 develops the formal foundations of canonical artifact identity, including the canonicalization pipeline and the canonical record structure. Section 3 specifies the satoshi-binding protocol and the satoshi handle construct. Section 4 introduces the provenance envelope architecture and its propagation semantics. Section 5 defines deterministic replay and reconstruction protocols. Section 6 presents the lifecycle governance framework for regulated institutional tokenization. Section 7 addresses regulatory and compliance considerations. Section 8 concludes and identifies forward directions for the series.

2 Canonical Artifact Identity: Foundations

2.1 Defining Canonicality

The term "canonical" derives from the concept of a canon — an authoritative rule or standard. In the context of digital artifact identity, a canonical artifact is the single authoritative representation of an artifact from which all other representations are derived and against which all representations are verified. This definition has precise technical content that must be distinguished from colloquial usage.

Definition 1 — Canonical Artifact

A **canonical artifact** is a digital record *A* such that: (i) *A* has a unique, deterministically computable serialization *S(A)* that is byte-identical for all conformant serializers regardless of implementation language, operating system, or execution environment; (ii) the SHA-256 digest *H = SHA256(S(A))* serves as the globally unique identifier for this specific version of *A*; and (iii) any party in possession of *H* and the schema definition for *A*'s record type can verify whether any presented artifact matches this canonical form by recomputing the digest.

Canonicality is emphatically not about file format. A PDF, a JSON object, and a structured database record can all represent the same underlying artifact, but none of these is canonical by virtue of its format. Canonicality is about *deterministic derivability*: given the artifact's content and a canonicalization protocol, every conformant implementation must produce the same byte sequence, and therefore the same digest. This is precisely the problem that the JSON Canonicalization Scheme (JCS), published as RFC 8785 in June 2020, was designed to solve.

RFC 8785 defines canonicalization of JSON data by building on the strict serialization methods for JSON primitives defined by ECMAScript, constraining JSON data to the Internet JSON (I-JSON) subset defined in RFC 7493, and applying deterministic property sorting by Unicode code point. The output of JCS is a "hashable" representation of JSON data suitable for use in cryptographic operations including hashing and digital signature. For institutional artifact identity, JCS provides the serialization layer on which canonical digests can be computed.

The choice of JSON as the canonical serialization substrate is deliberate. ISO 20022 financial messaging standards increasingly use JSON representations alongside their traditional XML encodings, and W3C Verifiable Credentials and Decentralized Identifiers are JSON-LD native. A JCS-based canonicalization

approach is therefore interoperable with the primary standards in use across institutional financial infrastructure.

An important property of canonical identity is **version isolation**: a canonical record represents a specific version of an artifact at a specific point in time. The canonical digest of version 1.0 of a trade confirmation and the canonical digest of version 1.1 of the same trade confirmation are distinct digests, and neither is derivable from the other. This means that amendment of an artifact necessarily produces a new canonical record with a new canonical digest — which is precisely the behavior required for an immutable, tamper-evident audit trail.

2.2 The Canonical Record Structure

The canonical record is the structured data object that serves as the authoritative representation of an artifact within the identity and provenance system. It is not the artifact itself — a trade confirmation remains a trade confirmation — but rather a structured metadata record that encodes the artifact's identity, its cryptographic digest, and sufficient contextual information to locate, verify, and interpret the artifact.

A canonical record contains the following fields, specified as a JCS-compatible JSON schema:

Field	Type	Description	Required
`schema_id`	URI string	Identifies the canonical record schema version and artifact type. Must be a versioned, resolvable URI pointing to the schema definition.	Yes
`subject_digest`	Hex string (64 chars)	SHA-256 digest of the JCS-serialized artifact payload. This is the primary identity reference for the artifact.	Yes
`artifact_type`	Enum string	Categorical type: TRADE_CONFIRMATION, BOND_INSTRUMENT, TITLE_DEED, REGULATORY_FILING, etc.	Yes
`identity_block`	Object	Encodes actor identity (DID or LEI), organization, agent (software/human), and delegation chain for the canonicalization authority.	Yes

Field	Type	Description	Required
`temporal_anchor`	Object	Block height and block hash of the Bitcoin block at or before which the canonical record was committed. Provides cryptographic timestamp resistant to retroactive manipulation.	Yes
`attestation_refs`	Array of objects	References to external attestation documents: SLSA provenance statements, in-toto link metadata, C2PA manifests, Sigstore/cosign transparency log entries, and regulatory filing references.	Conditional
`predecessor_digest`	Hex string (64 chars)	If this canonical record amends a prior record, this field contains the subject_digest of the immediately preceding canonical record, forming the provenance chain.	Conditional
`record_digest`	Hex string (64 chars)	SHA-256 digest of the JCS-serialized canonical record itself (excluding this field). Enables integrity verification of the record independent of the artifact payload.	Yes

The `identity_block` sub-structure supports the W3C Decentralized Identifiers (DID) specification for actor identification and the Legal Entity Identifier (LEI) standard for organizational identification. The two are not mutually exclusive — an institutional actor may be identified by both a DID (for cryptographic key binding) and an LEI (for legal entity recognition in regulatory contexts).

The `temporal_anchor` exploits the Bitcoin ledger's role as a global, Byzantine fault-tolerant timestamp authority. By recording the block height and block hash at the time of canonicalization, the canonical record establishes a lower bound on its creation time that cannot be forged without controlling the global hash rate. This is materially stronger than any timestamp issued by a single trusted timestamping authority.

2.3 From Real-World Artifact to Canonical Form

The canonicalization pipeline transforms an artifact from its raw operational form into a canonical record and its associated canonical digest. The pipeline consists of five deterministic stages, each of which must be auditable and reproducible:

1. **Normalization:** The raw artifact is transformed into a structured data representation, resolving format-specific ambiguities (e.g., PDF encoding artifacts, XML namespace aliasing, character encoding inconsistencies). Normalization is schema-specific and must be fully documented for each artifact type.
2. **Schema Validation:** The normalized artifact is validated against the versioned schema for its artifact type. Invalid artifacts are rejected before canonicalization; partial or ambiguous artifacts require human review and governance escalation.
3. **JCS Serialization:** The validated, normalized artifact is serialized using the JSON Canonicalization Scheme (RFC 8785). Object properties are sorted by Unicode code point; strings are escaped per ECMAScript rules; numbers are serialized in the IEEE 754 double-precision format. The output is a byte-stable string.
4. **Digest Computation:** SHA-256 is applied to the UTF-8 encoding of the JCS output string. The result is the subject digest — the artifact's canonical identity.
5. **Ledger Commitment:** The canonical record (containing the subject digest, identity block, temporal anchor, and attestation references) is constructed, JCS-serialized, its record digest computed, and the record is committed to the UTXO ledger via a binding transaction (described in Section 3).

The following worked example illustrates the pipeline applied to a bilateral trade confirmation:

WORKED EXAMPLE: Trade Confirmation Canonicalization Pipeline Step 1 — Raw Artifact: Input: PDF file (trade_conf_20260713_0147.pdf) Size: 84,231 bytes; contains structured trade data embedded as PDF form fields plus free-text annotations. Step 2 — Normalization: Extract structured fields: trade_id, counterparty_a_lei, counterparty_b_lei, instrument_isin, notional_amount, settlement_date, settlement_currency, trade_timestamp_utc. Resolve: Remove annotations; normalize numeric fields to fixed-point 8-decimal representation; convert all timestamps to ISO 8601 UTC format. Output: Structured JSON object

with 14 fields. Step 3 — Schema Validation: Schema URI: https://schemas.example-institution.com/ trade-confirmation/v2.3 Result: VALID — all required fields present, all type constraints satisfied. Step 4 — JCS Serialization: Apply RFC 8785 to the validated JSON object. Key ordering (alphabetical by Unicode code point): counterparty_a_lei → counterparty_b_lei → instrument_isin → notional_amount → ... → trade_timestamp_utc Output: 847-byte UTF-8 string (byte-stable). Step 5 — Digest Computation: subject_digest = SHA256(JCS_output) = "3a7f9b2c1e4d6f8a0b3c5e7d9f1a2b4c..." (64 hex characters) Step 6 — Ledger Commitment: Canonical record constructed; record_digest computed. Binding transaction submitted to Bitcoin network. TXID: "a1b2c3d4e5f6..." — confirmed at block height 904,217. temporal_anchor = { "height": 904217, "block_hash": "000000000000..." }

2.4 Canonical vs. Mutable Representations

A fundamental architectural discipline of the canonical identity system is the strict separation between canonical records, which are immutable and hash-locked, and operational representations, which are mutable, human-readable, and subject to amendment in the ordinary course of business. Conflating these two layers is the primary source of identity fragmentation in existing tokenization systems.

Operational representations serve the needs of business users: they may be formatted for human readability, localized into multiple languages, summarized for management reporting, or enriched with real-time market data. They are intentionally dynamic. Canonical records serve the needs of settlement, audit, and compliance: they are intentionally static, and once created, they must never be modified. Any modification to a canonical record — even the correction of a typographical error — would change the record digest and therefore invalidate all downstream references to that digest.

The update protocol resolves this tension cleanly. When an artifact is amended, the amendment does not modify the existing canonical record. Instead, a new canonical record is created for the amended

artifact, with its `predecessor_digest` field set to the `subject_digest` of the prior canonical record. The provenance envelope for the artifact receives an `AMENDMENT_REF` event referencing both digests. The result is a chain of canonical records — one per version of the artifact — in which each record is immutable and the chain is cryptographically linked. The complete amendment history is thus preserved without any violation of the immutability constraint.

2.5 Multi-Party Canonicalization

Many institutional artifacts are bilateral or multi-lateral: a trade confirmation is agreed between two counterparties; a syndicated loan agreement may involve ten or more parties. For such artifacts, the canonicalization protocol must address the challenge of achieving consensus on the canonical form before the artifact can be committed to the ledger.

The multi-party canonicalization protocol proceeds in three phases. In the **proposal phase**, one party (typically the initiating counterparty or a designated canonical authority) produces a proposed canonical record and broadcasts it to all other parties with a JCS-serialized, digitally signed proposal envelope. In the **review phase**, each party independently re-executes the canonicalization pipeline on its own representation of the artifact, computes the subject digest, and compares it to the proposed digest. Agreement is indicated by signing the proposal envelope; disagreement triggers a structured conflict resolution protocol. In the **commitment phase**, once the threshold of required signatures has been collected (typically all parties for bilateral agreements, or a governance-defined quorum for multi-lateral agreements), the canonical record is committed to the ledger via a binding transaction co-signed by all consenting parties.

Deterministic field ordering via JCS is essential to this protocol: it eliminates the possibility of disagreement arising from different parties' serializers ordering the same object fields differently. Schema versioning ensures that all parties are operating against the same field definitions and type constraints. Conflict resolution protocols define the escalation path when parties produce different subject digests — typically involving a neutral canonical authority designated in the master agreement between the parties.

3 The Satoshi-Binding Protocol

3.1 Why Satoshis as Identity Anchors

Section 1.3 introduced the satoshi as an identity anchor candidate. This section revisits those properties in greater technical depth and contrasts the satoshi-based approach with alternatives that have been proposed or deployed in other tokenization frameworks.

The UTXO model of the Bitcoin protocol has a property that is seldom foregrounded in general discussions of blockchain technology but is critical for identity anchoring: **each UTXO is a discrete, uniquely identified, non-duplicable object**. A UTXO is identified by the tuple (TXID, output_index), and at any moment it is either in the UTXO set (unspent) or it is not. There is no mechanism by which the same UTXO can appear in two nodes' UTXO sets simultaneously in a consistent network state. This discreteness and uniqueness is the foundational property that makes UTXO-native satoshis suitable as identity anchors.

Account-based ledgers, by contrast, record balances rather than discrete UTXO objects. A balance of 1,000 units in an account is not traceable to specific originating transactions; the units are indistinguishable from one another; and the account model does not natively support the concept of "this specific unit of value is the one that carries the identity of artifact X." Attempts to impose token-level identity on account-based ledgers require additional off-chain indexing layers, which reintroduce the trust and availability risks that the ledger was intended to eliminate.

Within the UTXO model, individual satoshis have an additional property that is exploitable for identity anchoring: their **coinbase-traceable lineage**. Using ordinal tracking methodologies (and their institutional variants), every satoshi in the UTXO set can be assigned a unique ordinal number based on its coinbase issuance order. This ordinal is invariant across the satoshi's entire history — it never changes regardless of how many transactions the satoshi passes through. An artifact's identity can therefore be bound not merely to a UTXO (which is ephemeral — it is consumed and recreated with each transaction) but to a specific satoshi ordinal, which is permanent.

3.2 Binding Protocol Mechanics

The satoshi-binding protocol creates a mutual, cryptographically verifiable link between a canonical artifact record and a specific satoshi (or set of satoshis) in the UTXO set. The protocol is "mutual" in the sense that the artifact digest is embedded in the binding transaction, and the binding transaction's TXID becomes embedded in the canonical record — each references the other, creating a closed, tamper-evident loop.

The binding protocol proceeds through the following steps:

6. **Canonical Digest Preparation:** The canonical record is finalized (all fields populated, all required signatures collected) and the record_digest is computed. The subject_digest is the primary input to the binding transaction.
7. **Binding Transaction Construction:** A Bitcoin transaction is constructed with at least two outputs: (a) an `OP_RETURN` output encoding the subject_digest as a 32-byte push data payload, establishing the ledger-embedded digest reference; and (b) a satoshi-carrying output locked to the artifact's designated custody address, with a locking script that may include additional constraints encoding the artifact's governance rules (e.g., multi-signature requirements, timelock conditions).
8. **Satoshi Handle Recording:** After the binding transaction is confirmed, the satoshi handle is computed as the tuple (TXID, output_index, satoshi_offset) where TXID is the confirmed transaction ID, output_index identifies the satoshi-carrying output, and satoshi_offset identifies which specific satoshi within that output carries the artifact identity (relevant for outputs carrying multiple satoshis, where ordinal-level tracking is employed).
9. **Canonical Record Finalization:** The canonical record is updated with the satoshi handle and the temporal anchor (block height and block hash of the confirming block), and the final record_digest is computed. The finalized canonical record is distributed to all parties and stored in the provenance envelope.
10. **Binding Verification:** Any party can verify the binding by: (i) locating the OP_RETURN output in the binding transaction and extracting the embedded digest; (ii) comparing it to the subject_digest in the canonical record; (iii) confirming that the canonical record's satoshi handle

correctly references the binding transaction's satoshi-carrying output; and (iv) confirming that the temporal anchor matches the confirming block's height and hash.

Definition 2 — Satoshi Handle

A **satoshi handle** is the tuple *(TXID, output_index, satoshi_offset)* that uniquely and persistently identifies the satoshi(s) carrying a specific artifact's canonical identity within the UTXO set. The TXID and output_index identify the UTXO containing the identity-carrying satoshi(s); the satoshi_offset, expressed as an ordinal within the output's value range, identifies the specific satoshi at the sub-UTXO level. The satoshi handle persists as the artifact's primary ledger identity reference throughout its lifecycle, updated at each transfer event to reflect the new UTXO location of the identity-carrying satoshi(s).

3.3 The Satoshi Handle and Identity Persistence

The satoshi handle is the artifact's permanent ledger identity reference. Its critical property is that it persists across transfers: when the UTXO containing the identity-carrying satoshi is spent in a transfer transaction, the satoshi moves to a new UTXO in the new owner's custody address, and the satoshi handle is updated to reference the new UTXO while the satoshi_offset (if ordinal-tracked) remains unchanged. The identity of the artifact thus travels with its satoshi regardless of which wallet, custody account, or institutional address holds it at any moment.

This persistence across transfers is what distinguishes satoshi-bound identity from transaction-bound identity (where identity is tied to a specific TXID, which is consumed at each transfer) and from address-bound identity (where identity is tied to a specific Bitcoin address, which changes at each transfer in best-practice key management). The satoshi handle, updated at each transfer event in the provenance envelope, provides a continuous, unbroken identity thread from issuance through archival.

The ledger traversal protocol for reconstructing the satoshi handle history begins from the current UTXO containing the identity-carrying satoshi and traces backward through the UTXO lineage by following the input chain of each transaction: the current UTXO was created by a transaction whose inputs consumed prior UTXOs; those UTXOs were created by earlier transactions; and so on, back to the original binding

transaction. This traversal is fully deterministic given the UTXO set and transaction graph, and requires no off-chain data to complete.

3.4 Selective Binding and Fungibility Partitioning

A conceptual subtlety arises at the interface between the institutional identity system and the Bitcoin protocol's native fungibility properties. At the protocol level, all satoshis of equal denomination are fungible — any satoshi can satisfy any payment obligation of the appropriate amount. However, for institutional settlement purposes, satoshis that have been bound to a canonical artifact record are operationally non-fungible: they must not be mixed with unbound satoshis or satoshis bound to different artifacts, because doing so would sever the identity chain.

The mechanism for enforcing this operational non-fungibility is institutional, not protocol-level. Custody systems that participate in the canonical identity and provenance framework implement **fungibility partitioning**: a tagging and tracking layer that maintains, for each UTXO in custody, a record of which satoshis within that UTXO are identity-carrying and which artifacts they are bound to. Transactions that would commingle identity-carrying and non-identity-carrying satoshis are blocked at the custody layer before being broadcast to the network.

This approach preserves the Bitcoin protocol's integrity — no protocol-level changes are required — while enabling institutional-grade identity tracking at the satoshi level. The partition is enforced by institutional policy and technical controls, not by the consensus rules, which is appropriate: the consensus rules establish the security of the ledger, while institutional governance establishes the operational disciplines that run above it.

3.5 Binding Integrity and Collision Resistance

The cryptographic security of the satoshi-binding protocol rests on two foundational properties: the collision resistance of SHA-256 and the unforgeability of Bitcoin digital signatures (ECDSA or Schnorr over the secp256k1 curve).

SHA-256 collision resistance ensures that it is computationally infeasible to find two distinct artifact payloads that produce the same subject_digest. As of 2026, no practical SHA-256 collision has been

found, and the current best theoretical attacks require computational resources far beyond any institutional adversary's reach. The use of SHA-256 — the same hash function that secures Bitcoin's proof-of-work mechanism — also ensures that the canonical identity system benefits from the extensive cryptanalytic scrutiny that SHA-256 has received over more than two decades.

The infeasibility of forging a binding can be stated precisely: to claim that artifact *A′* is bound to a satoshi that is actually bound to artifact *A*, an adversary would need to either (i) find a SHA-256 collision (infeasible), (ii) forge the digital signature on the binding transaction (infeasible without the private key), or (iii) rewrite the confirmed transaction in the ledger (infeasible without controlling more than 50% of the network's hash rate for a sustained period). Any one of these would require a capability that would simultaneously compromise the security of the entire Bitcoin network — a dependency that institutional risk frameworks can reasonably treat as acceptable.

Temporal anchoring provides an additional layer of binding integrity: because the canonical record's temporal anchor references a specific block height and block hash, any attempt to claim a retroactive binding — asserting that an artifact was bound to a satoshi before the actual binding transaction — can be immediately refuted by checking the ledger, which shows no OP_RETURN output containing the artifact's digest at the claimed prior block height.

3.6 Cross-Chain and Cross-Ledger Binding Considerations

Institutional tokenization ecosystems frequently involve multiple ledgers: the originating UTXO ledger (Bitcoin), one or more EVM-compatible chains, private permissioned ledgers operated by individual institutions, and central bank digital currency infrastructure. A complete identity and provenance architecture must address how satoshi-bound identity interacts with these other environments.

The core principle is **identity reference, not identity duplication**. When a satoshi-bound artifact is represented on another ledger — for example, as a wrapped token on an EVM chain, or as a balance entry in a private permissioned ledger — the representation on the other ledger carries a *reference* to the satoshi handle and the canonical record on the originating UTXO ledger, but does not replicate the binding itself. The canonical binding remains singular and located on the originating UTXO ledger; all other representations are secondary, derivative, and explicitly subordinated to the canonical identity.

This architecture prevents the fragmentation of canonical identity across ledgers — a risk that would otherwise allow multiple parties to assert competing "canonical" bindings on different ledgers. The canonical record's `schema_id` and `subject_digest` serve as the cross-ledger reference keys, enabling any party on any ledger to look up the canonical record on the originating UTXO ledger and verify the binding without requiring any intermediary trust.

4 Provenance Envelopes

4.1 The Provenance Envelope Architecture

The canonical record, as defined in Section 2, is the immutable identity snapshot of an artifact at a specific version. However, institutional settlement requires more than identity snapshots — it requires a complete, ordered, verifiable history of every event that has affected the artifact since its creation. This history is the **provenance envelope**.

Definition 3 — Provenance Envelope

A **provenance envelope** is the structured, append-only metadata container that travels with a canonical artifact throughout its complete lifecycle. It accumulates a cryptographically ordered record of every state transition, custody transfer, encumbrance, amendment reference, regulatory filing, and attestation that the artifact undergoes from issuance through archival. The provenance envelope is not itself the canonical record — the canonical record is immutable and version-specific — but rather the artifact's complete operational biography, verifiable end-to-end against the originating canonical record and the UTXO ledger.

The distinction between the canonical record and the provenance envelope is architecturally critical and must be maintained with discipline. The canonical record answers the question "what is this artifact?" The provenance envelope answers the question "what has happened to this artifact?" Both are necessary for regulated settlement; neither alone is sufficient.

4.2 Envelope Schema

The provenance envelope has a defined hierarchical schema consisting of five primary components:

Component	Structure	Description
Envelope Header	Object (fixed)	Schema version, artifact canonical digest (links envelope to canonical record), creation timestamp, issuing institution DID/LEI, and envelope_id (UUID v4).
Event Log	Ordered array (append-only)	Chronologically ordered array of signed state transition events. Each event is individually signed by its authoring party. The array is append-only — events are never removed or reordered.
Attestation Bundle	Array of attestation objects	External proofs and attestations: SLSA provenance statements, in-toto link metadata, Sigstore/cosign transparency log entries, C2PA manifests (for media artifacts), and regulatory filing references.
Custody Chain	Ordered sequence	Ordered record of custody holders with their DID/LEI identifiers, custody start and end timestamps, satoshi handle at time of receipt, and cryptographic acknowledgment signatures.
Integrity Seal	Object (versioned)	SHA-256 digest of the complete envelope state at each checkpoint, signed by the sealing party. Seals are accumulating — each new seal references the prior seal's digest, forming a chain of integrity checkpoints.

4.3 State Transition Events

The event log is the heart of the provenance envelope. Each entry in the event log represents a state transition that the artifact has undergone. The event types are formally enumerated and each has a defined payload schema:

Event Type	Trigger Condition	Required Payload Fields
`ISSUANCE`	First binding transaction confirmed on ledger.	canonical_digest, satoshi_handle, issuer_did, binding_txid, block_height
`TRANSFER`	Satoshi-carrying UTXO spent in a transfer transaction.	prior_satoshi_handle, new_satoshi_handle, transferor_did, transferee_did, transfer_txid, timestamp

Event Type	Trigger Condition	Required Payload Fields
`ENCUMBRANCE`	Artifact placed under a legal encumbrance (pledge, repo, lien).	encumbrance_type, beneficiary_did, encumbrance_terms_digest, effective_timestamp, expiry_timestamp
`AMENDMENT_REF`	Artifact amended; new canonical record created.	prior_canonical_digest, new_canonical_digest, amendment_authority_did, rationale_digest, effective_timestamp
`REDEMPTION`	Artifact reaches its settlement endpoint; final value delivered.	redemption_value, redemption_currency, redeemer_did, settlement_txid, settlement_timestamp
`SUSPENSION`	Artifact temporarily suspended from circulation (regulatory hold, dispute).	suspension_authority_did, suspension_reason_code, effective_timestamp, expected_resolution_timestamp
`TERMINATION`	Artifact lifecycle permanently concluded (post-redemption or cancellation).	termination_authority_did, termination_reason_code, final_satoshi_handle, archive_reference

Every event, regardless of type, carries four mandatory meta-fields: `event_type`, `event_timestamp` (ISO 8601 UTC), `actor_did` (the DID of the party recording the event), and `actor_signature` (ECDSA or Schnorr signature by the actor's key over the JCS-serialized event payload). The satoshi handle reference at the time of the event is also included where applicable, enabling direct cross-reference to the ledger state at each event point.

4.4 Envelope Propagation

Provenance envelopes propagate through institutional infrastructure as the artifact moves through its lifecycle. The propagation model is designed to ensure that every party that receives an artifact also receives the complete provenance envelope, and that each party's interactions with the artifact are recorded in the envelope before it is forwarded onward.

The envelope update protocol at each custody hop proceeds as follows: the receiving party validates the incoming envelope by (i) verifying the integrity seal chain against the current event log, (ii) verifying all event signatures in the event log, and (iii) verifying that the envelope header's canonical_digest matches the canonical record for the artifact being received. Having validated the envelope, the receiving party

records its custody acknowledgment in the custody chain, appends any events that occurred during its custody period to the event log, computes a new integrity seal over the updated envelope, and forwards the updated envelope to the next party in the chain.

The envelope update protocol is designed to be incremental: each hop adds only new events and a new integrity seal; it does not require re-processing the entire event history. This makes the protocol computationally efficient for long-lived artifacts with extensive event histories.

4.5 Selective Disclosure and Privacy

Regulated institutional contexts frequently require selective disclosure: a party may need to share the provenance history of an artifact with a regulator or counterparty without revealing events that are confidential to other parties (e.g., events involving third-party counterparties who have not consented to disclosure).

Selective disclosure is enabled by organizing the event log as a Merkle tree over the ordered array of event entries. Each event is a leaf node; the Merkle root is incorporated into the integrity seal. A party seeking to share a subset of events can produce a **redacted provenance disclosure** that includes: (i) the selected events in their complete, signed form; (ii) Merkle proofs for each selected event demonstrating that it is included in the event log at its claimed position; and (iii) the integrity seal (including the Merkle root) signed by the sealing party. The recipient can verify that the disclosed events are authentic and correctly positioned in the event log without seeing the withheld events.

This selective disclosure mechanism satisfies both the verifiability requirement (the regulator can confirm the disclosed events are authentic) and the confidentiality requirement (events involving third parties are not disclosed without authorization). It is directly analogous to the selective disclosure mechanisms specified in the W3C Verifiable Credentials specification for credential attributes.

4.6 Envelope Anchoring

Provenance envelopes are maintained off-chain as signed data structures held by the parties involved in the artifact's lifecycle. However, for long-horizon verifiability and tamper-evidence, envelopes are periodically anchored to the Bitcoin ledger through an envelope anchoring transaction.

At defined checkpoints (e.g., at each significant lifecycle event, or on a scheduled periodic basis), the current integrity seal of the provenance envelope is committed to the ledger in an OP_RETURN output within a transaction that also references the artifact's satoshi handle. This creates a timestamped, ledger-anchored checkpoint of the envelope state — a proof that the envelope contained its current event set at or before the checkpoint block height. Any subsequent modification of the event log prior to the checkpoint would be detectable as a seal mismatch. The anchoring transaction thus extends the tamper-evidence of the UTXO ledger to encompass the off-chain provenance envelope, providing a cryptographic guarantee of envelope integrity that is independent of the envelope holder's own infrastructure.

5 Deterministic Replay and Reconstruction

5.1 The Reconstruction Requirement

Regulated financial markets impose on participants an obligation that goes beyond maintaining current records: they must be able to reconstruct, with regulatory-grade certainty, the precise state of any instrument, account, or transaction at any historical point in time. This obligation arises from multiple regulatory drivers operating simultaneously.

Post-trade audit requirements under EMIR (Article 9), SFTR (Article 4), and MiCA (Articles 76–80) mandate that firms maintain complete, accurate, and time-stamped records of all transactions, reportable for at least five years (or longer for certain instrument categories). SEC Rule 17a-4 imposes electronic record-keeping requirements on broker-dealers and investment advisers that include non-erasability, indexing, and availability for immediate regulatory examination. Basel III and IV supervisory frameworks require banks to demonstrate the completeness and accuracy of their position records at any requested historical timestamp during regulatory stress examination.

Dispute resolution in post-trade settlement frequently hinges on the precise state of an instrument at a specific moment. When two counterparties disagree on whether a settlement obligation was satisfied, the resolution depends on establishing — to the satisfaction of a neutral arbitrator or court — what the state of the instrument was at the claimed settlement time. This establishment is only possible if the

instrument's history is encoded in an immutable, ordered, canonical sequence of events from which that historical state can be recovered deterministically.

5.2 Replay Semantics

Definition 4 — Replay Semantics

Replay is the deterministic process of applying the ordered event sequence from a provenance envelope to the initial canonical record of an artifact to derive the artifact's state at any target point in time *t*. Replay is deterministic in the following strict sense: given the same initial canonical record and the same ordered, integrity-verified event log, every honest, conformant replay engine will derive the same artifact state at any given target timestamp, regardless of when the replay is performed, by whom, or on what infrastructure.

Determinism of replay is the technical foundation of institutional trust in the canonical identity system. It means that the artifact's state at any historical moment is not a matter of assertion by any party — it is a matter of computation from publicly verifiable inputs. No party can unilaterally claim that an artifact was in a particular state at a particular time; the claim must be verifiable by replay from the canonical record and the integrity-sealed event log.

The replay process proceeds by: (i) loading the initial canonical record; (ii) loading the integrity-verified event log from the provenance envelope; (iii) iterating through the event log in chronological order, applying each event's state transition to the current artifact state; (iv) halting when the target timestamp is reached or the last event before the target timestamp has been applied; and (v) returning the resulting artifact state as the reconstructed historical state.

For replay to be deterministic, state transition functions for each event type must be deterministic — the same event applied to the same prior state must always produce the same resulting state. This requirement places constraints on the event payload schema: state transition functions may not reference external data sources (e.g., market data feeds, external clocks) that could produce different

outputs at different replay times. All relevant external data must be embedded in the event payload at event recording time.

5.3 The Replay Engine

The replay engine is the software component that implements the replay protocol. Its architecture is designed around a single overriding principle: the replay engine must be stateless. A stateless replay engine consumes its inputs (canonical record + event log) and produces its outputs (historical artifact state) without maintaining any persistent internal state between invocations. This statelessness has two critical implications: first, any party can run an independent replay engine without depending on any shared infrastructure; second, the replay engine's outputs can be independently verified by running a second instance against the same inputs and confirming output equality.

The replay engine consists of the following components:

- **Event Log Parser:** Validates the event log's integrity seal chain, verifies all event signatures, and deserializes the event array into an ordered sequence of typed event objects.
- **State Machine:** Implements the defined state transition function for each event type. Maintains the current in-memory artifact state during replay, updating it as each event is applied. The state machine's transition graph must be fully specified and versioned as part of the schema definition.
- **Canonical Record Resolver:** Retrieves and verifies the initial canonical record by subject_digest, including resolution of any predecessor chain for artifacts that have been amended prior to the replay target timestamp.
- **Satoshi Handle Verifier:** Cross-references the satoshi handle sequence in the event log against the UTXO ledger, confirming that each recorded satoshi movement corresponds to an actual, confirmed ledger transaction.
- **Output State Validator:** Applies the schema definition to the reconstructed state to confirm that it is a valid instance of the artifact type at the target timestamp.

5.4 Reconstruction from Ledger State

A particularly important capability of the canonical identity system is the ability to reconstruct the provenance history of an artifact from the UTXO ledger alone, without access to the off-chain provenance envelope. This capability is relevant in disaster recovery scenarios (where a party's envelope storage has been destroyed), in regulatory examination (where the regulator seeks an independent verification of the reported provenance history), and in insolvency proceedings (where the administrator may not have access to the insolvent party's envelope storage).

Ledger-only reconstruction proceeds via the **ledger traversal protocol**. Beginning from the current satoshi handle (which can be determined by inspecting the current UTXO set for outputs that reference the artifact's canonical digest in their OP_RETURN companion outputs), the protocol traverses backward through the UTXO lineage:

11. Identify the current UTXO containing the identity-carrying satoshi.
12. Find the transaction that created this UTXO; its inputs identify the prior UTXO(s).
13. Examine the OP_RETURN output (if any) in this transaction for provenance event data.
14. Repeat steps 2–3 for each prior UTXO, back to the original binding transaction.
15. Reconstruct the event sequence from the OP_RETURN payloads collected during the traversal.

The ledger traversal protocol recovers only those events that were explicitly anchored to the ledger (via OP_RETURN outputs or envelope anchoring transactions). Off-chain events that were recorded in the provenance envelope but not ledger-anchored will not be recoverable from the ledger alone. This is by design: off-chain events represent a higher-resolution record that is maintained by the parties involved, while the ledger record represents the lower-resolution, universally accessible backup. The system is designed so that the most significant lifecycle events (ISSUANCE, TRANSFER, REDEMPTION, TERMINATION) are always ledger-anchored, while minor operational events (e.g., internal custody bookkeeping) may be envelope-only.

5.5 Partial Reconstruction and Gap Handling

In practice, provenance envelopes may have gaps — periods during which events occurred but were not recorded or anchored. Gaps can arise from: a party failing to record events before becoming insolvent;

off-chain events that were never forwarded to the envelope holder; ledger anchoring failures due to network congestion; or deliberate omission by a malicious party.

The replay engine's gap handling protocol distinguishes between three categories of gap:

- **Detectable Anchoring Gaps:** Periods between envelope anchoring checkpoints where the ledger shows no anchoring transaction for the artifact. These gaps are detectable by comparing the event log timestamps against the anchoring transaction timestamps. Events within an anchoring gap are flagged as unanchored; their integrity is supported only by the event signatures, not by ledger anchoring.
- **Missing Event Gaps:** Periods where the event log shows no events but ledger activity (e.g., UTXO movements) indicates that events should have been recorded. These gaps are detected by the Satoshi Handle Verifier comparing the ledger UTXO history against the event log. Missing event gaps trigger a gap report, which must be resolved through governance escalation before reconstruction can be certified as complete.
- **Unresolvable Gaps:** Gaps that cannot be filled by any available data source. In these cases, the replay engine applies a conservative state estimation — reporting the most recently verified state before the gap and flagging all states derived from post-gap events as provisional until the gap is resolved or formally accepted as irresolvable through the governance framework.

5.6 Replay in Dispute Resolution: A Worked Example

The following worked example illustrates how replay semantics provide a deterministic resolution mechanism for post-settlement disputes. Consider a bilateral bond settlement between Institution A (the seller) and Institution B (the buyer):

> **Dispute Scenario:** Institution A asserts that the bond instrument was transferred to Institution B at 14:32:07 UTC on 10 July 2026, settling the trade. Institution B asserts that the settlement was not complete until 14:47:22 UTC on the same date, due to a failure in the initial transfer transaction. The difference matters

because an ENCUMBRANCE event recorded by a third party at 14:40:00 UTC would be either pre-settlement or post-settlement depending on which assertion is correct.

Resolution via replay proceeds as follows: Both parties submit their provenance envelopes to the neutral replay engine. The engine first verifies that both envelopes share the same envelope_id and canonical_digest (confirming they relate to the same artifact). It then constructs the merged event log by taking the union of both envelopes' event logs and verifying that no events are in conflict (same event type at same timestamp with different payloads) — if conflicts exist, the ledger-anchored version of conflicting events takes precedence. The merged event log is sorted chronologically and replayed from the initial canonical record. At 14:32:07 UTC, the event log shows a TRANSFER event signed by Institution A referencing transfer TXID "a1b2..."; the Satoshi Handle Verifier confirms this transaction was confirmed at block height 904,201, with a block timestamp of 14:32:11 UTC. At 14:40:00 UTC, the ENCUMBRANCE event is applied. At 14:47:22 UTC, no second TRANSFER event appears in the merged event log. The replay engine's output is unambiguous: the TRANSFER was complete at 14:32:11 UTC (ledger confirmation time), and the ENCUMBRANCE event at 14:40:00 UTC is post-settlement. The ruling is deterministic, cryptographically verifiable by any party with access to the canonical record, the merged event log, and the UTXO ledger, and cannot be overridden by any party's unilateral assertion.

6 Lifecycle Governance for Institutional Tokenization

6.1 Lifecycle Phases

A satoshi-bound artifact passes through five defined lifecycle phases from creation through archival. Each phase has specific governance requirements, authorized actors, and required ledger events:

#	Phase	Description	Key Events	Terminal Condition
1	**Pre-Issuance**	Artifact creation, canonicalization pipeline execution, multi-party consensus on canonical form, and binding transaction construction.	None (pre-ledger)	Binding transaction broadcast to network.

#	Phase	Description	Key Events	Terminal Condition
2	**Issuance**	Binding transaction confirmed on ledger; ISSUANCE event recorded in provenance envelope; satoshi handle established; envelope distributed to initial custody holder.	ISSUANCE	Envelope distributed and custody acknowledged.
3	**Active Circulation**	Artifact under active institutional use — subject to transfers, encumbrances, and amendments. Provenance envelope accumulates events; periodic ledger anchoring occurs.	TRANSFER, ENCUMBRANCE, AMENDMENT_REF, SUSPENSION	REDEMPTION event recorded.
4	**Settlement**	Final value delivery; REDEMPTION event recorded; satoshi-carrying UTXO transferred to settlement agent or burned to provably unspendable output.	REDEMPTION	Settlement confirmed on ledger.
5	**Archival**	Post-redemption envelope sealing; TERMINATION event recorded; final integrity seal computed and committed to ledger; envelope transferred to long-term archive.	TERMINATION	Archive receipt confirmed.

6.2 Governance Rules per Phase

Each lifecycle phase has a defined governance matrix specifying who can authorize state transitions, what proofs are required, and what ledger and regulatory actions must occur:

In the **Pre-Issuance** phase, the Issuer (assisted by the Canonical Authority) is the sole authorized party for canonicalization and binding transaction construction. All counterparties must provide their consensus signatures before the binding transaction is broadcast. No ledger events occur during this phase, but all pre-issuance records (normalization logs, schema validation results, consensus signatures) are captured in the attestation bundle that will be incorporated in the provenance envelope at Issuance.

In the **Issuance** phase, the ISSUANCE event must be signed by the Issuer and attested by the Canonical Authority. Regulatory notifications required under applicable frameworks (e.g., trade repository reporting under EMIR; position reporting under MiCA) must be triggered within the regulatory reporting

window, which varies by jurisdiction and instrument type. The binding transaction confirmation block height and hash are committed as the temporal anchor.

In the **Active Circulation** phase, TRANSFER events require signatures from both the transferor (releasing custody) and the transferee (accepting custody). ENCUMBRANCE events require signatures from the beneficial owner and the encumbrance beneficiary. AMENDMENT_REF events require the amendment governance threshold defined in the master agreement between the parties. SUSPENSION events may be initiated unilaterally by a Regulator Observer (with the appropriate jurisdiction overlay) but must be acknowledged by the Custodian within a defined period.

In the **Settlement** phase, the REDEMPTION event requires signatures from the Settlement Agent and the final beneficial owner. The settlement transaction on the UTXO ledger must be confirmed before the REDEMPTION event is recorded in the provenance envelope. Cross-referencing the settlement TXID against the ledger is a mandatory step in the REDEMPTION event validation.

In the **Archival** phase, the TERMINATION event is recorded by the Issuer or its designated successor, and the final integrity seal is computed and committed to the ledger. The sealed envelope is transferred to the designated long-term archive, which must confirm receipt with a cryptographic acknowledgment. The archive reference is recorded in the TERMINATION event payload.

6.3 Amendment Governance

The amendment protocol for canonical artifacts is one of the most governance-sensitive elements of the lifecycle framework, because it must balance two competing requirements: the immutability of the canonical record (which is the source of the system's integrity guarantees) and the operational reality that institutional artifacts are frequently amended during their lifecycle (which is a basic commercial and legal necessity).

The protocol resolves this tension by recognizing that immutability and amendment are not in conflict if they operate at different levels of the architecture. The **canonical record is immutable at the version level**: once a canonical record has been committed to the ledger, its content cannot be changed without invalidating its record_digest, which would be immediately detectable. **Amendment operates at the artifact level**: when an artifact is amended, a new canonical record is created for the amended version,

with the `predecessor_digest` field referencing the prior version's subject_digest. The provenance envelope receives an AMENDMENT_REF event referencing both versions.

This approach creates a linked list of canonical records — one per artifact version — where each record is individually immutable and the links between records are cryptographically sealed. The complete amendment history is preserved without modification of any existing record. Any party can reconstruct the full amendment chain by following the predecessor_digest links from the current canonical record back to the original, and can verify that no links have been inserted or removed by checking the AMENDMENT_REF events in the provenance envelope against the canonical record chain.

Amendment authorization thresholds are defined in the master agreement governing the artifact. Minor amendments (e.g., correction of formatting errors, addition of informational fields) may require signatures from a single designated party; substantive amendments (e.g., changes to notional amount, settlement date, or governing law) require signatures from all counterparties and, in some cases, regulatory notification.

6.4 Multi-Jurisdiction Governance

Institutional artifacts frequently operate across multiple legal jurisdictions simultaneously. A cross-border trade between a European investment bank and a US broker-dealer may be subject to both EMIR and SEC reporting requirements; a tokenized bond issued in one jurisdiction and traded in several others may need to comply with MiCA in the EU, FCA regulations in the UK, and applicable securities laws in Asia-Pacific markets. The governance framework must accommodate this multi-jurisdictional reality without creating conflicting or duplicative identity structures.

The architecture addresses multi-jurisdictional governance through the **jurisdiction overlay** — a structured component that is layered on the provenance envelope and records jurisdiction-specific events, regulatory filings, and compliance attestations without modifying the core canonical identity structure. The jurisdiction overlay is keyed by jurisdiction code (e.g., EU-EMIR, US-SEC, UK-FCA) and accumulates a separate event log and attestation bundle for each relevant jurisdiction. Each jurisdiction overlay is independently sealed and can be selectively disclosed to the relevant regulatory authority without revealing other jurisdictions' overlays.

The single canonical record and provenance envelope serve as the neutral identity backbone that all jurisdiction overlays reference. This means that despite the multi-jurisdictional complexity, there is only one canonical identity for the artifact — not one per jurisdiction — and all regulatory disclosures are derived from and verifiable against the same underlying canonical record. This is the "neutral identity layer" property described in Section 7.4.

6.5 Institutional Role Framework

The lifecycle governance system defines six institutional roles, each with specific permissions relative to the canonical record and provenance envelope:

Role	Primary Responsibility	Canonical Record Permissions	Provenance Envelope Permissions
Issuer	Creates and commits the initial canonical record; initiates the provenance envelope; executes the binding transaction.	Create (initial only); Read	Create; Append (ISSUANCE, TERMINATION); Read
Canonical Authority	Attests to the correctness of the canonicalization pipeline; resolves multi-party canonicalization conflicts; validates amendment records.	Attest; Read	Attest; Read; Integrity Seal
Custodian	Holds the satoshi-carrying UTXO; records TRANSFER and ENCUMBRANCE events; maintains envelope integrity during custody period.	Read	Append (TRANSFER, ENCUMBRANCE); Custody Chain Update; Integrity Seal
Verifier	Independently verifies canonical record integrity and provenance envelope completeness; operates replay engine for dispute resolution.	Read; Verify	Read; Verify; Replay
Regulator Observer	Cryptographically privileged read access to jurisdiction-relevant provenance envelope events; no modification rights; may initiate SUSPENSION events within jurisdiction.	Read (within jurisdiction scope)	Read (selective disclosure); Append (SUSPENSION, jurisdiction scope only)
Settlement Agent	Executes final value delivery; records REDEMPTION event; coordinates the archival transition.	Read	Append (REDEMPTION); Coordinate TERMINATION

6.6 Archival and Long-Term Verifiability

Regulated financial institutions are required to maintain records for periods of five to thirty or more years, depending on instrument type and jurisdiction. The canonical identity system must therefore be designed not merely for current operational use but for long-horizon verifiability — the ability to reconstruct and verify artifact history decades after the artifact's lifecycle has concluded.

Long-horizon verifiability faces two primary challenges: cryptographic obsolescence (hash functions and signature algorithms that are secure today may be deprecated or broken in twenty years) and infrastructure evolution (the specific software systems used to manage envelopes today will be replaced multiple times over a thirty-year horizon).

The system addresses cryptographic obsolescence through a **hash function migration protocol**. When SHA-256 is deprecated, the migration protocol specifies: (i) a transition period during which canonical records are re-attested using the new hash function without invalidating the original SHA-256 digests; (ii) a dual-digest period during which both the original and new digests are maintained in parallel; and (iii) a legacy reference period during which the original digests are preserved as historical references alongside the new digests. The UTXO ledger's OP_RETURN payloads from prior anchoring transactions remain permanently accessible as the ground-truth reference, providing a tamper-evident backbone that is independent of the specific hash function used at any moment.

The UTXO ledger's role as the persistent, tamper-evident backbone is the long-horizon verifiability guarantee that no off-chain system can match: as long as the Bitcoin ledger is operational (which, by any reasonable institutional planning assumption over a thirty-year horizon, it will be), the binding transactions and envelope anchoring transactions committed to it remain immutable and accessible, providing the cryptographic anchors against which all claims about artifact history can be independently verified.

7 Regulatory and Compliance Considerations

7.1 Alignment with Post-Trade Regulation

The canonical identity and provenance architecture described in this paper was designed from the outset to satisfy the record-keeping, reporting, and audit trail requirements of the principal post-trade regulatory frameworks applicable to institutional financial instruments. The following alignment analysis addresses the four most significant frameworks for the primary institutional markets where this architecture is expected to be deployed.

EMIR (EU) 648/2012: Article 9 of EMIR requires that both counterparties to an OTC derivative report their transactions to a registered trade repository, including a unique trade identifier and all material terms. The canonical record's subject_digest serves as a cryptographically unique trade identifier that is more robust than LEI-generated UTIs because it is derived from the artifact's content rather than assigned by a registration authority. The provenance envelope's event log satisfies EMIR's requirement for reporting lifecycle events (amendments, terminations) in a time-stamped, ordered format.

SFTR (Securities Financing Transactions Regulation): SFTR imposes analogous reporting requirements for securities financing transactions (repos, securities lending, margin loans). The canonical record's structured field schema is directly mappable to SFTR reporting fields, and the provenance envelope's TRANSFER and ENCUMBRANCE events correspond directly to the lifecycle events reportable under SFTR. The ledger-anchored temporal anchors satisfy SFTR's timestamp accuracy requirements without dependence on a single trusted time source.

MiCA Articles 76–80: MiCA's provisions on operational and security risk management for asset-referenced token issuers include requirements for data integrity, auditability, and record-keeping. The canonical identity system's immutability guarantees and Merkle-verified integrity seals directly satisfy MiCA's data integrity requirements. The BSV Association's MiCA white paper (2026) provides additional guidance on how UTXO-native token infrastructure maps to MiCA compliance obligations, and the architecture described in this paper is consistent with that guidance.

SEC Rule 17a-4: This rule requires broker-dealers to maintain electronic records in a non-erasable, non-rewriteable format (WORM) for specified periods and to make them immediately available for regulatory examination. The canonical record's immutability (enforced by its ledger-committed record_digest) satisfies the non-erasable, non-rewriteable requirement; the replay engine's ability to reconstruct artifact state at any historical timestamp satisfies the immediate availability requirement;

and the selective disclosure mechanism enables immediate access for regulatory examination without requiring disclosure to other parties.

7.2 AML/KYC Integration

Anti-money laundering (AML) and know-your-customer (KYC) compliance in institutional tokenization contexts has historically relied on self-reported identity data, supplemented by periodic third-party verification. The canonical identity system enables a qualitatively stronger approach: AML/KYC compliance checks can be performed against cryptographically attested identity claims that are embedded in the canonical record and provenance envelope.

The `identity_block` of every canonical record includes the DID and/or LEI of the canonicalization authority and, where applicable, the delegating institution. DIDs are cryptographically bound to public keys — a party claiming a DID can prove control of that DID by signing a challenge with the corresponding private key. LEIs are issued by accredited Local Operating Units (LOUs) following due diligence on the legal entity. Both identity types are therefore verifiable claims rather than self-assertions.

The provenance envelope's custody chain records the DID and/or LEI of each custody holder, with their cryptographic acknowledgment signature at each transfer. An AML screening system can traverse the custody chain and screen each actor against sanctions lists and adverse media databases, with the confidence that the screened identities are cryptographically attested rather than self-reported. This approach is compatible with the W3C Verifiable Credentials framework, enabling institutional identity claims to be presented and verified using standard VC verification protocols.

7.3 Supervisory Access

The Regulator Observer role defined in Section 6.5 provides the technical mechanism for supervisory access to provenance envelope data. Supervisory access is implemented through a cryptographic capability grant — the Issuer or Canonical Authority issues a signed capability token to the relevant regulatory authority, scoped to the jurisdiction overlay corresponding to the authority's jurisdiction. The

capability token enables the authority to decrypt and read the events in that jurisdiction overlay without requiring disclosure of events in other jurisdiction overlays or in the core envelope.

This mechanism satisfies two competing requirements simultaneously: the regulatory authority's right to access information relevant to its jurisdiction (satisfied by the capability grant and the jurisdiction overlay), and the institution's obligation to protect confidential information from disclosure to parties not entitled to receive it (satisfied by the scoping and encryption of jurisdiction overlays). The mechanism is analogous to the selective disclosure approach described in Section 4.5, applied at the jurisdiction-overlay level.

Supervisory access is read-only by default. The Regulator Observer role's one exception — the ability to append SUSPENSION events within its jurisdiction scope — is constrained by a governance rule requiring that SUSPENSION events be acknowledged by the Custodian within a defined period, ensuring that the institution is notified of any regulatory holds and can respond appropriately.

7.4 Cross-Border Recognition

A persistent challenge in cross-border financial regulation is the absence of a neutral identity layer that is recognized by all relevant jurisdictions without depending on any single jurisdiction's recognition authority. National security identifiers (ISINs, CUSIPs) are issued by jurisdiction-specific numbering agencies and their recognition is governed by bilateral or multilateral agreements. LEIs are more universal but still require registration with an accredited LOU. Neither provides a cryptographically self-evident identity that can be verified independently of any registration authority.

Satoshi-bound canonical identity is, by construction, cryptographically self-evident and jurisdiction-neutral. The canonical record's subject_digest is derived from the artifact's content using an open, standardized algorithm (SHA-256 over JCS-serialized JSON); it does not require registration with any authority; it can be verified by any party with access to the artifact, the canonical record, and a SHA-256 implementation. The UTXO ledger binding is verifiable by any party with access to the Bitcoin UTXO set, which is publicly accessible worldwide without any jurisdictional restriction.

This jurisdiction-neutrality makes satoshi-bound canonical identity particularly valuable in cross-border settlement contexts: two institutions in different jurisdictions can establish, with cryptographic

certainty, that they are referring to the same artifact version without relying on any jurisdiction-specific authority to certify that equivalence. The canonical digest is the universal reference. Each jurisdiction's regulatory requirements are then satisfied through the jurisdiction overlay mechanism, which adds jurisdiction-specific attestations on top of the neutral canonical identity without modifying the identity itself.

8 Conclusion

This paper has presented a complete, technically rigorous, and institutionally viable identity and provenance layer for tokenized assets in regulated settlement contexts. The architecture rests on five mutually reinforcing contributions, each of which is necessary but insufficient alone.

Canonical artifact identity — grounded in RFC 8785 JCS, SHA-256 digest binding, and the canonical record structure — establishes the technical meaning of "the authoritative form of this artifact," resolving the provenance ambiguity that plagues current tokenization systems and providing the immutable identity anchor on which all downstream components depend.

The satoshi-binding protocol — exploiting the atomicity, persistent UTXO lineage, non-duplicability, and observable state-change properties of individual satoshis — anchors canonical artifact identity to the most tamper-evident, globally accessible, Byzantine fault-tolerant ledger infrastructure currently in existence, providing identity persistence across transfers, splits, and custody movements that no account-based or record-keeping alternative can match.

The provenance envelope architecture — with its append-only event log, Merkle-verified integrity seals, jurisdiction overlays, and selective disclosure mechanisms — transforms the satoshi-bound canonical identity into a complete operational biography of the artifact, satisfying the record-keeping, audit trail, and selective disclosure requirements of regulated institutional finance across multiple jurisdictions simultaneously.

Deterministic replay and reconstruction semantics — implemented by the stateless replay engine with satoshi handle verification and gap handling protocols — provide the guarantee that no historical state claim about any artifact can be asserted unilaterally; every claim is verifiable by any party from the canonical record and event log, providing a cryptographic foundation for dispute resolution, regulatory examination, and insolvency administration that is superior to any purely record-based alternative.

Lifecycle governance — spanning pre-issuance through archival, with defined governance rules for each phase, a clean amendment protocol, multi-jurisdiction overlays, and a formal institutional role framework — provides the institutional structure within which the technical architecture operates, ensuring that the system's cryptographic guarantees are matched by operational disciplines that are auditable, enforceable, and compliant with the regulatory obligations of participating institutions.

The central claim of this paper is stated plainly in its abstract and is now substantiated in full: regulated settlement systems require not just value transfer but **deterministic identity transfer** — the guarantee that the cryptographic identity of the artifact being settled accompanies every ledger event through its complete lifecycle. The architecture described in this paper delivers that guarantee at institutional scale, using open standards, publicly accessible ledger infrastructure, and formally specified protocols that any conformant implementation can independently verify.

The road ahead in this series is equally demanding. Paper IV, *Custody Infrastructure Integration for Satoshi-Bound Assets*, will address how the canonical identity and provenance layer described here integrates with institutional custody systems — qualified custodian frameworks, multi-party computation key management, and the interface between UTXO-native custody and traditional securities account structures. Paper V, *Cross-Network Settlement Protocols and Atomic Multi-Party Finality*, will address the construction of cross-ledger settlement protocols that preserve canonical identity across network boundaries while achieving atomic finality guarantees. Together, the five papers in this series constitute a complete architectural specification for blockchain-native institutional settlement infrastructure suitable for deployment in regulated financial markets.

References

- **[1]** Rundgren, A., Jordan, B., and Erdtman, S. (2020). *RFC 8785: JSON Canonicalization Scheme (JCS)*. IETF Independent Submission. June 2020. Available at: https://www.rfc-editor.org/rfc/rfc8785
- **[2]** Rose, S., Borchert, O., Mitchell, S., and Connelly, S. (2020). *NIST Special Publication 800-207: Zero Trust Architecture*. National Institute of Standards and Technology, U.S. Department of Commerce. August 2020.
- **[3]** ISO (2013). *ISO 20022: Financial Services — Universal Financial Industry Message Scheme*. International Organization for Standardization. Geneva, Switzerland. (Continuously maintained; current edition 2022.)

- **[4]** Open Source Security Foundation (OSSF) (2023). *SLSA: Supply Chain Levels for Software Artifacts Framework, Version 1.0*. Available at: https://slsa.dev/spec/v1.0
- **[5]** Torres-Arias, S., et al. (2019). *in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes*. USENIX Security Symposium. Specification maintained at: https://in-toto.io
- **[6]** Coalition for Content Provenance and Authenticity (C2PA) (2024). *C2PA Technical Specification, Version 2.1: Provenance Standard for Digital Content*. Joint Development Foundation. Available at: https://c2pa.org/specifications/specifications/2.1/specs/C2PA_Specification.html
- **[7]** European Parliament and Council (2023). *Regulation (EU) 2023/1114 of the European Parliament and of the Council on Markets in Crypto-Assets (MiCA)*. Official Journal of the European Union. 9 June 2023.
- **[8]** European Parliament and Council (2012). *Regulation (EU) No 648/2012 of the European Parliament and of the Council on OTC Derivatives, Central Counterparties and Trade Repositories (EMIR)*. Official Journal of the European Union. 4 July 2012.
- **[9]** U.S. Securities and Exchange Commission (2003). *Rule 17a-4: Records to be Preserved by Certain Exchange Members, Brokers and Dealers*. 17 CFR § 240.17a-4. As amended.
- **[10]** World Wide Web Consortium (W3C) (2022). *Decentralized Identifiers (DIDs) v1.0: Core Architecture, Data Model, and Representations*. W3C Recommendation. 19 July 2022. Available at: https://www.w3.org/TR/did-core/
- **[11]** World Wide Web Consortium (W3C) (2022). *Verifiable Credentials Data Model v1.1*. W3C Recommendation. 3 March 2022. Available at: https://www.w3.org/TR/vc-data-model/
- **[12]** Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. Self-published. October 2008. Available at: https://bitcoin.org/bitcoin.pdf
- **[13]** BSV Association (2026). *Bitcoin SV and the Markets in Crypto-Assets Regulation (MiCA): A Compliance Framework for UTXO-Native Token Infrastructure*. BSV Association White Paper Series. January 2026.
- **[14]** European Parliament and Council (2015). *Regulation (EU) 2015/2365 of the European Parliament and of the Council on Transparency of Securities Financing Transactions and of Reuse (SFTR)*. Official Journal of the European Union. 23 December 2015.

- **[15]** Financial Stability Board (2023). *Tokenisation in Financial Markets: Regulatory Considerations and Policy Approaches*. FSB Report to the G20. October 2023. Available at: https://www.fsb.org
- **[16]** Bank for International Settlements (2023). *BIS Working Paper No. 1141: Tokenisation on Permissioned and Permissionless Blockchains: A Framework for Institutional Markets*. BIS Monetary and Economic Department. Basel, Switzerland.
- **[17]** Sigstore Project (2023). *Sigstore and Cosign: Artifact Signing and Transparency for Software Supply Chains*. Technical Specification v2.0. Linux Foundation. Available at: https://docs.sigstore.dev
- **[18]** Birkholz, H., Delignat-Lavaud, A., Fournet, C., Guy, S., and Laskar, S. (2023). *RFC 9162: Certificate Transparency Version 2.0*. IETF. December 2023. (Referenced as transparency log design precedent for envelope anchoring.)
- **[19]** European Central Bank (2024). *New Technologies for Wholesale Settlement: Findings from the ECB Wholesale DLT Trials*. ECB Occasional Paper. Frankfurt am Main. September 2024.
- **[20]** Corda, R7 (2023). *Deterministic Transaction Finality and State Machine Semantics in Distributed Ledger Platforms*. R3 Technical White Paper Series. (Referenced for comparative analysis of state machine replay in enterprise DLT contexts.)